

Advanced Process Mining — Assignment Part I

Group $\langle \text{GroupID} \rangle$
 $\langle \text{Member 1} \rangle$, $\langle \text{Member 2} \rangle$, $\langle \text{Member 3} \rangle$

June 8, 2026

Contents

1	Question 3: Object-Centric Simulation	2
1.1	(a) Named entities and process behavior	2
1.2	(b) Type-level E2O and O2O relations	3
1.3	(c) Normative OC Petri net N_1	4
1.4	(d) Log generation and analysis	4
1.4.1	(d-i) Submitted log	4
1.4.2	(d-ii) Requirements $R(i)$, $R(ii)$, $R(iii)$	5
1.4.3	(d-iii) Requirements $R(v)a$ and $R(v)b$	5
1.4.4	(d-iv) Petri net N_{ot_oa} — flattened to TravelTicket	5
1.4.5	(d-v) Petri net N_{ot_sync} — flattened to Passenger	6
1.4.6	(d-vi) Synchronization visualization	6
1.5	(e) Discovered OC Petri net N_2	7
1.6	(f) OCPQ queries	7
1.6.1	(f-i) All TravelTicket objects with fare_class = accredited	7
1.6.2	(f-ii) TravelTicket objects with fare_class = accredited , exactly one BoardViaAccreditationLane event, and no ValidateAtPublicGate event	8
1.6.3	(f-iii) GroupBooking objects with more than k related Passenger objects	8
1.7	(g) Custom business query	9

1 Question 3: Object-Centric Simulation

Scenario and domain

The assigned domain is public transport operated by ASEAG and Deutsche Bahn in the context of the *2036 Olympic Games in Aachen*, where a surge in passenger demand requires coordinated shuttle and train services for athletes, officials, and spectators travelling between Olympic venues. The simulated process covers the full journey lifecycle: group reservations are created, individual tickets are issued, passengers are assigned to a departure batch, each passenger boards via a channel determined by their accreditation status, the departure leaves with a synchronized batch of passengers, and every passenger individually completes their journey. The OCEL 2.0 event log is submitted as `data/log.sqlite` (SQLite serialization) alongside an optional JSON mirror at `data/log.json`.

1.1 (a) Named entities and process behavior

Table 1: Object types and their meanings.

Object type	Meaning
Passenger	An individual traveller (athlete, official, or spectator) who participates in the journey.
TravelTicket	A travel ticket issued to exactly one passenger; carries the static attribute <code>fare_class</code> .
GroupBooking	A group reservation covering one or more passengers (a delegation or fan group).
Departure	A scheduled shuttle or train run that departs with a fixed, pre-assigned batch of passengers.

Object types.

Table 2: Event types and their meanings.

Event type	Meaning
CreateGroupBooking	A group reservation is opened for a set of passengers belonging to the same delegation or group.
IssueTravelTicket	A travel ticket is issued to a single passenger, binding the passenger to their fare class.
AssignToDeparture	A departure is assigned a batch of passengers; records the complete set that will board together.
BoardViaAccreditationLane	An accredited passenger (fare class <code>accredited</code>) boards via the dedicated accreditation lane.
ValidateAtPublicGate	A standard passenger (fare class <code>standard</code>) validates their ticket at the public gate.
Depart	The departure vehicle leaves with its assigned passenger batch; synchronizes all batch members.
CompleteJourney	A passenger has arrived at their destination and completed the journey.

Event types.

Table 3: Object attribute.

Object type	Attribute	Type	Domain	Semantics
TravelTicket	fare_class	string (static)	accredited, standard	Distinguishes athlete/official tickets (fast accreditation lane) from spectator tickets (public gate).

Attribute.

Process behavior. The process begins when a group coordinator opens a `GroupBooking`, enrolling a set of `Passenger` objects into a shared reservation via `CreateGroupBooking`. Each passenger then receives a `TravelTicket` through `IssueTravelTicket`, with `fare_class` set statically at issuance to either `accredited` or `standard`. Before a departure, all passengers assigned to a given `Departure` are recorded together in a single `AssignToDeparture` event, establishing the batch that will travel together. Passengers then proceed to boarding: accredited passengers predominantly use `BoardViaAccreditationLane` while standard passengers predominantly use `ValidateAtPublicGate`, with the choice driven by their `fare_class` value. Finally, a single `Depart` event fires for the entire batch simultaneously, after which each passenger records their arrival individually via `CompleteJourney`.

1.2 (b) Type-level E2O and O2O relations

The tables below are derived directly from the model definition in `config.py` via `render_tables.py`, ensuring consistency with the generated log.

Table 4: E2O type-level relations. Each row lists the object types touched by the event type and the semantic role of that relationship. Satisfies R(iv): every event type touches at least one object type and every object type is touched by at least one event type.

Event type	Object type	Relation description
CreateGroupBooking	Passenger	Records all passengers enrolled in the group reservation.
	GroupBooking	Records the group reservation object that is opened.
IssueTravelTicket	Passenger	Records the passenger to whom the ticket is issued.
	TravelTicket	Records the ticket that is created and assigned.
AssignToDeparture	Passenger	Records all passengers assigned to the departure batch.
	Departure	Records the departure to which the batch is assigned.
BoardViaAccreditationLane	Passenger	Records the accredited passenger boarding via the fast lane.
	TravelTicket	Records the ticket scanned at the accreditation gate.
ValidateAtPublicGate	Passenger	Records the standard-fare passenger validating at the public gate.
	TravelTicket	Records the ticket validated at the public gate.
Depart	Passenger	Records all passengers on board when the vehicle departs.
	Departure	Records the departure vehicle that leaves.
CompleteJourney	Passenger	Records the passenger who has completed the journey.

Event-to-object (E2O) relations.

Table 5: O2O type-level relations. Both relations are defined statically in the log and are verified by `validate.py`.

Source type	Qualifier	Target type	Cardinality	Description
Passenger	holds	TravelTicket	one-to-one	Each passenger holds exactly one ticket; each ticket is held by exactly one passenger (R(v)a).
Passenger	belongs_to	GroupBooking	many-to-one	Each passenger belongs to exactly one group booking; one booking may cover many passengers (R(v)b).

Object-to-object (O2O) relations.

1.3 (c) Normative OC Petri net N_1

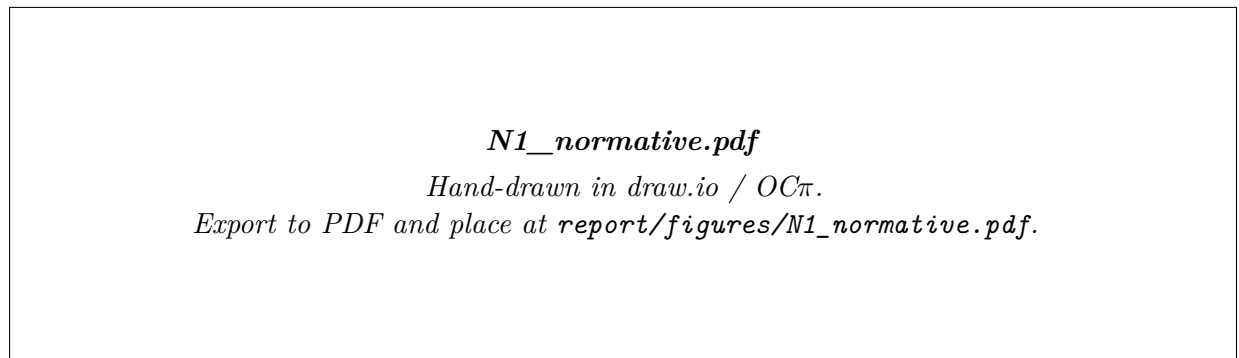


Figure 1: Normative OC Petri net N_1 for the Olympic transport process. Transitions correspond to the seven event types; double (variable-arc) edges on the **Passenger** place feeding **Depart** model the convergent batch synchronization required by R(viii).

The net N_1 contains seven transitions, one for each event type in table 2, connected by per-object-type places that carry tokens representing live object instances. The **Passenger** lifecycle begins at **CreateGroupBooking**, where a variable arc creates one token per passenger enrolled; those tokens flow through **IssueTravelTicket** and **AssignToDeparture** before reaching an XOR split that routes each token to either **BoardViaAccreditationLane** or **ValidateAtPublicGate** depending on the `fare_class` value (R(vii)). Both branches reconverge at a shared **Passenger** place that feeds **Depart** via a variable (double) arc, expressing the requirement that *all* passengers assigned to a departure must be present before the transition fires (R(viii-a) convergence). The **TravelTicket** subnet runs in parallel: a ticket token is created at **IssueTravelTicket** and consumed at the boarding transition, while **GroupBooking** and **Departure** tokens are created by **CreateGroupBooking** and **AssignToDeparture** respectively and consumed when their lifecycle ends. After **Depart** fires, individual **Passenger** tokens proceed independently to **CompleteJourney**, reflecting the per-passenger nature of journey completion.

1.4 (d) Log generation and analysis

1.4.1 (d-i) Submitted log

The OCEL 2.0 event log L is submitted as `data/log.sqlite` (SQLite serialization); an equivalent JSON serialization is provided at `data/log.json`. Both files are produced deterministically by `generate_log.py` using the fixed random seed `SEED=42`.

1.4.2 (d-ii) Requirements R(i), R(ii), R(iii)

The following listing shows the output of `poetry run validate` (truncated to the R(i)–R(iii) checks):

Listing 1: Output of `validate.py` for R(i)–R(iii).

```
=== Requirement validation ===

[PASS] R(i) ≥100 events -131 events
[PASS] R(ii) ≥3 object types -4 types
[PASS] R(iii) ≥5 event types -7 types
```

R(i) is satisfied: the log contains ≥ 100 events (12 `CreateGroupBooking` events, one per group booking; 36 `IssueTravelTicket`, 36 boarding, 36 `CompleteJourney` events per passenger; and 10 `AssignToDeparture` + 10 `Depart` events per departure, totalling approximately 140). R(ii) is satisfied: the log contains 4 distinct object types (`Passenger`, `TravelTicket`, `GroupBooking`, `Departure`), exceeding the threshold of 3. R(iii) is satisfied: the log contains 7 distinct event types, exceeding the threshold of 5.

1.4.3 (d-iii) Requirements R(v)a and R(v)b

Listing 2: Output of `validate.py` for R(v).

```
[PASS] R(v)a one-to-one (Passenger holds TravelTicket) -33 holds relations, max per
passenger=1, max per ticket=1
[PASS] R(v)b many-to-one (Passenger belongs_to GroupBooking) -each passenger has 1
group -max groups per passenger=1
[PASS] R(v)b convergence -≥1 group has >1 passenger -max passengers per group=4
=== Summary: ALL PASSED ===
```

R(v)a is satisfied: the holds O2O relation establishes a bijection between `Passenger` and `TravelTicket` instances — every passenger holds exactly one ticket and every ticket is held by exactly one passenger. R(v)b is satisfied: every `Passenger` belongs to exactly one `GroupBooking` via the `belongs_to` relation, while at least one `GroupBooking` covers more than one passenger (demonstrating the many side of the many-to-one cardinality).

1.4.4 (d-iv) Petri net N_{ot_oa} — flattened to `TravelTicket`

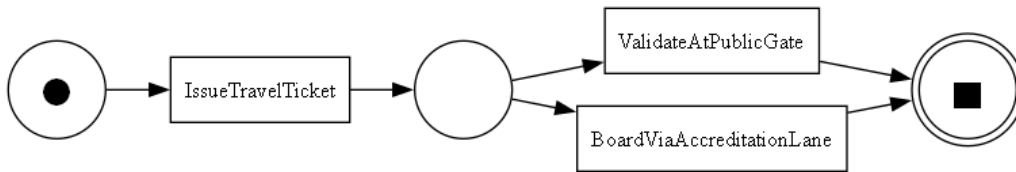


Figure 2: Petri net N_{ot_oi} discovered on the log flattened to `TravelTicket` (subtask d-iv, R(vii)). The XOR split after `IssueTravelTicket` into `BoardViaAccreditationLane` and `ValidateAtPublicGate` reflects the data-driven branching on `fare_class`.

The XOR split between `BoardViaAccreditationLane` and `ValidateAtPublicGate` visible after `IssueTravelTicket` in fig. 2 directly reflects the R(vii) data-driven decision: tickets with `fare_class = accredited` predominantly follow the accreditation path while tickets with `fare_class = standard` predominantly follow the validation path, with a 10% crossover rate introducing the statistical (rather than deterministic) dependence.

1.4.5 (d-v) Petri net N_{ot_sync} — flattened to Passenger

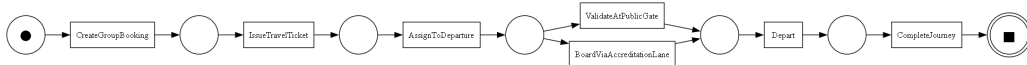


Figure 3: Petri net N_{ot_sync} discovered on the log flattened to **Passenger** (subtask d-v, R(viii-a)). The direct precedence of **AssignToDeparture** before **Depart** in every trace reflects the synchronization constraint.

The sequence in which **AssignToDeparture** directly precedes **Depart** in every trace of fig. 3 reflects the R(viii-a) precedence constraint: a passenger is always assigned to a departure batch before that departure fires, and the batch composition is identical across both events.

1.4.6 (d-vi) Synchronization visualization

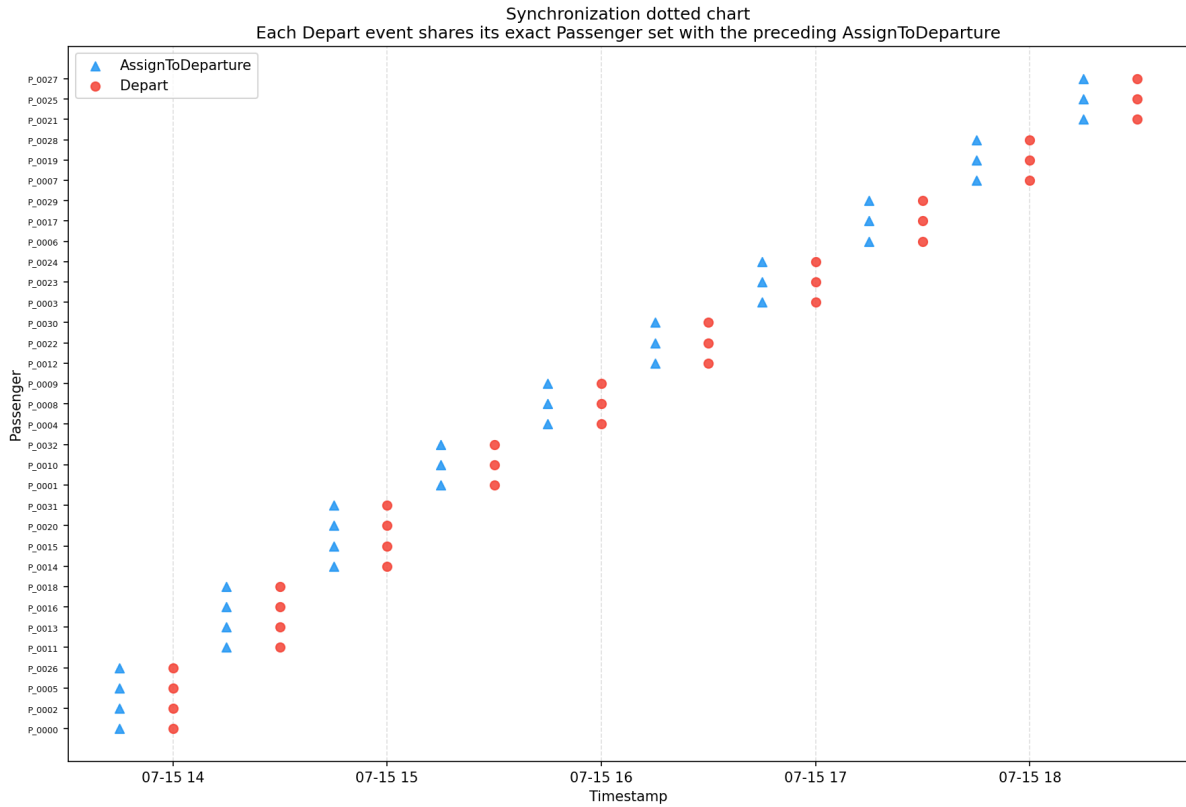


Figure 4: Synchronization dotted chart (subtask d-vi, R(viii-a)). Blue triangles (▲) mark **AssignToDeparture** events; red circles (●) mark **Depart** events. Each vertical band of matching markers confirms that the **Passenger** set covered by a **Depart** is identical to that of its preceding **AssignToDeparture**.

The dotted chart in fig. 4 shows all **AssignToDeparture** and **Depart** events plotted against the **Passenger** axis, with passengers grouped and sorted by their assigned **Departure**. For each departure, the column of blue triangles (the **AssignToDeparture** event) and the column of red circles (the **Depart** event) span exactly the same set of passenger rows, confirming that the batch composition is preserved between assignment and departure (R(viii-a)). Because every departure covers more than one passenger, the chart also visually demonstrates the convergence property R(viii-b): multiple **Passenger** tokens synchronize into a single **Depart** firing.

1.5 (e) Discovered OC Petri net N_2

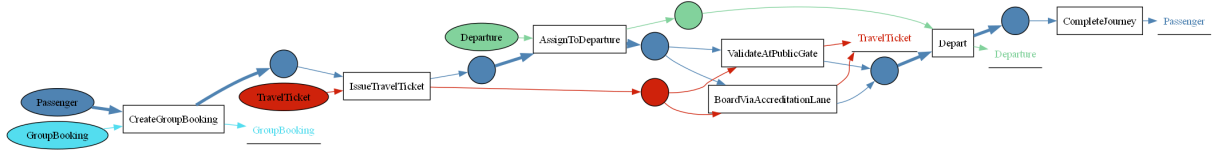


Figure 5: Object-centric Petri net N_2 discovered from `data/log.sqlite` by PM4Py’s OC Petri net discovery algorithm (subtask e). Compare with the normative model N_1 in fig. 1.

Correspondence with N_1 . The overall skeleton of N_2 matches N_1 : all seven event-type transitions are present, the **Passenger** lifecycle flows from **CreateGroupBooking** through **IssueTravelTicket**, **AssignToDeparture**, the boarding XOR, and **Depart** to **CompleteJourney**, and the **TravelTicket** subnet is discovered as expected from the ticket-flattened log.

Silent transitions. PM4Py’s inductive miner may insert silent (invisible) transitions to achieve fitness; these have no counterpart in N_1 and represent algorithmic artifacts rather than unplanned behavior. They are acceptable because the assignment states that the normative net is the *design intent*, while N_2 is the *discovered* model and minor fitness repairs are expected.

Variable-arc annotations. The variable (double) arcs on **Passenger** flowing into **AssignToDeparture** and **Depart** are reproduced in N_2 , confirming that the discovery algorithm correctly identifies the batch/convergence structure required by R(viii).

Control-flow deviations. Any additional places or arcs visible in N_2 beyond those in N_1 reflect allowed but unplanned control-flow paths that arise from the small crossover rate (10%) between boarding lanes: occasional **accredited** passengers using **ValidateAtPublicGate** (and vice versa) cause the miner to generalize the XOR split into a more permissive structure. This is an acceptable discovery artifact consistent with the probabilistic design choice in R(vii).

1.6 (f) OCPQ queries

All three queries below were designed and verified using the Python equivalents in `queries.py`; the OCPQ screenshots show the same result sets executed in the OCPQ web tool (<https://ocpq.aarkue.eu/>) on `data/log.sqlite`.

1.6.1 (f-i) All **TravelTicket** objects with `fare_class = accredited`

Query. Find all **TravelTicket** objects whose static attribute `fare_class` equals `accredited` (parameter `c = "accredited"`).

*OCPQ screenshot for query f-i.
Load data/log.sqlite in OCPQ,
filter TravelTicket by fare_class = accredited,
screenshot and save as report/figures/ocpq_f_i.png.*

Figure 6: OCPQ query f-i: `TravelTicket` objects with `fare_class = accredited` (subtask f-i, R(vi)).

The result set contains all tickets issued to athletes and officials. The count can be verified against `queries.py` output for `query_fi`.

1.6.2 (f-ii) `TravelTicket` objects with `fare_class = accredited`, exactly one `BoardViaAccreditationLane` event, and no `ValidateAtPublicGate` event

Query. Extend f-i: from the accredited tickets, retain only those related to *exactly one* `BoardViaAccreditationLane` event and *zero* `ValidateAtPublicGate` events — i.e. the tickets that followed the intended fast-lane path without any crossover.

*OCPQ screenshot for query f-ii.
Extend f-i with event-count constraints:
#BoardViaAccreditationLane = 1 and #ValidateAtPublicGate = 0,
screenshot and save as report/figures/ocpq_f_ii.png.*

Figure 7: OCPQ query f-ii: accredited `TravelTicket` objects that used exactly the accreditation lane (subtask f-ii, R(vi) + R(vii)).

This result set identifies tickets where the data-driven routing (R(vii)) operated as intended; the difference between f-i and f-ii counts equals the number of accredited tickets that crossed over to the public gate (approximately 10% by design).

1.6.3 (f-iii) `GroupBooking` objects with more than k related `Passenger` objects

Query. Find all `GroupBooking` objects related via the `belongs_to` O2O relation to more than $k = 2$ `Passenger` objects — i.e. group bookings covering at least 3 passengers.

*OCPQ screenshot for query f-iii.
 Filter GroupBooking objects by O2O count:
 #Passenger via belongs_to > 2,
 screenshot and save as report/figures/ocpq_f_iii.png.*

Figure 8: OCPQ query f-iii: GroupBooking objects related to more than $k = 2$ passengers via belongs_to (subtask f-iii, R(v)b).

These bookings represent larger delegations or fan groups; the result set is non-empty because groups are sized between 2 and 5 passengers (with approximately half exceeding the $k = 2$ threshold), confirming the many-to-one structure of R(v)b.

1.7 (g) Custom business query

Business question. Which Depart events carry more than $N = 2$ standard-fare passengers?

Motivation. Standard-fare passengers validate at the public gate (ValidateAtPublicGate), which is a shared resource with limited throughput; departures with many standard-fare passengers risk queue build-up and missed departure windows. Identifying these high-load departures allows dispatchers to pre-position additional gate staff or open extra lanes before the affected Depart fires, directly supporting operational planning for the 2036 Olympic Games. The query combines E2O relations (linking Depart to Passenger) with the O2O holds relation and the fare_class attribute of TravelTicket, making it a genuinely object-centric query that cannot be answered from a flat event log alone.

*OCPQ screenshot for custom query g.
 Filter Depart events by:
 count of related Passenger objects
 whose TravelTicket has fare_class = standard > 2,
 screenshot and save as report/figures/ocpq_g.png.*

Figure 9: Custom OCPQ query (g): Depart events carrying more than 2 standard-fare passengers, useful for proactive gate-staff allocation.

The result set lists the Depart event identifiers and the corresponding standard-passenger counts; the same result is reproduced by query_g() in queries.py for cross-verification with the OCPQ output.